

Algoritmos e Estrutura de Dados

Aula 01 – Tipos básicos de dados:
Estruturas compostas (homogêneas
e heterogêneas).



Estruturas Compostas Homogêneas

Estrutura Compostas Homogêneas

São aquelas que armazenam elementos do **mesmo** tipo de dado. Isso significa que **todos** os elementos dentro da estrutura possuem o mesmo tipo, garantindo uma **alocação de memória uniforme** e facilitando o acesso aos dados. **Exemplos mais conhecidos:**

1. Array
2. Matriz (Array Bidimensional)
3. Vetor Dinâmico

Array

Valores

5	3	1	9
---	---	---	---

Posição

0	1	2	3
---	---	---	---

Matriz

Posição

0	1	2	3
---	---	---	---

Posição

0
1

5	3	1	9
2	4	7	6

Valores

Vetor Dinâmico

Valores

0x741598	0x258741	0x582791	0x357951
----------	----------	----------	----------

Posição

0	1	2	3
---	---	---	---

Estrutura Compostas Homogêneas

Vetor

```
#include <stdio.h>

int main(){

    int numeros[5]= {1, 2, 3, 4, 5}; // Array de inteiros

    char letras[3]= {'A', 'B', 'C'}; // Array de caracteres

    printf("Primeiro número: %d\n", numeros[0]);

    printf("Primeira letra: %c\n", letras[0]);


    return 0;

}
```

Array				
Valores	5	3	1	9
Posição	0	1	2	3

Estrutura Compostas Homogêneas

Matrizes

```
#include <stdio.h>
```

```
int main(){
```

```
    int matriz[2][3] = {{1, 2, 3}, {4, 5, 6}}; // Matriz 2x3
```

```
    printf("Elemento da primeira linha e segunda coluna: %d\n", matriz[0][1]);
```

```
    return 0;
```

```
}
```

		Matriz			
Posição		0	1	2	3
Posição	0	5	3	1	9
	1	2	4	7	6

Valores

Estrutura Compostas Homogêneas

Vetores Dinâmicos (malloc)

```
#include <stdio.h>

#include <stdlib.h>

int main(){

    int *vetor;

    int tamanho = 5;

    vetor = (int *) malloc(tamanho * sizeof(int));

    if (vetor == NULL){

        printf("Erro de alocação de memória!\n");

        return 1;

    }

    /* .... Continua */
```

```
/* .... Continua */

for(int i = 0; i < tamanho; i++){

    vetor[i] = i * 2;

    printf("%d ", vetor[i]);

}

free(vetor); // Liberando a memória alocada

return 0;

}
```

Vetor Dinâmico

Valores

Posição

0x741598	0x258741	0x582791	0x357951
0	1	2	3

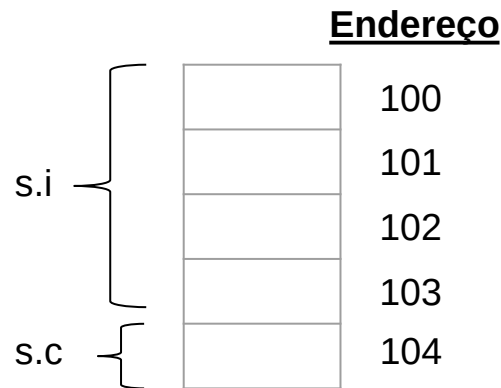
Estruturas Compostas Heterogêneas

Estrutura Compostas Heterogêneas

Pode ser definida como um conjunto de variáveis de **tipos diferentes** organizadas sob um único nome. Em programação, criar uma **"estrutura"** equivale à criação de um **novo tipo de dado**. Exemplo:

- **Struct ou Registro**

```
typedef struct {  
    int i;  
    char c;  
} Simple s;
```



Declarando Variáveis STRUCT

Exemplo1

```
struct Funcionario
```

```
{
```

```
    int idade;
```

```
    char nome[40];
```

```
    float salario;
```

```
};
```

```
struct Funcionario empregado;
```

```
struct Funcionario chefe;
```

```
struct Funcionario secretaria;
```

Exemplo2

```
struct Funcionario
```

```
{
```

```
    int idade;
```

```
    char nome[40];
```

```
    float salario;
```

```
} empregado, chefe, secretaria;
```

Declarando Variáveis STRUCT com Typedef

Exemplo1

```
struct Funcionario  
{  
    int idade;  
    char nome[40];  
    float salario;  
};  
  
typedef struct Funcionario Func;  
Func secretaria; //Declara uma variável do tipo Func
```

Exemplo2

```
typedef struct Funcionario  
{  
    int idade;  
    char nome[40];  
    float salario;  
} Func;  
  
Func secretaria; //Declara uma variável do tipo Func
```

Declarando Variáveis STRUCT com Typedef

Exemplo3

```
typedef struct
```

```
{
```

```
    int idade;
```

```
    char nome[40];
```

```
    float salario;
```

```
} Func;
```

```
Func secretaria; //Declara uma variável do tipo Func
```

```
Func funcionarios[10]; //Declara um vetor de struct funcionarios
```

Como manipular STRUCT's em C

Para acessar os itens de uma variável struct, vamos utilizar o **operador ponto (.)**

Exemplo: **minhaStruct.item**

chefe.idade; //é um inteiro como outro qualquer.
empregado.nome; //é uma string como outra qualquer.
secretaria.salario; //é um float como outro qualquer

Como inicializar STRUCT's em C

Exemplo1

```
typedef struct
```

```
{
```

```
    int idade;
```

```
    char nome[40];
```

```
    float salario;
```

```
} Func;
```

```
Func secretaria = {40, "Maria", 2500.00};
```

```
Func chefe = {45, "Jose", 12500.00};
```

Exemplo2

```
typedef struct
```

```
{
```

```
    int idade;
```

```
    char nome[40];
```

```
    float salario;
```

```
} Func;
```

```
Func chefe;
```

```
chefe.idade = 45;
```

```
strcpy(chefe.nome, "Jose");
```

```
chefe.salario = 12500.00;
```

Como inicializar STRUCT's em C

Exemplo3

```
typedef struct
```

```
{
```

```
    int idade;
```

```
    char nome[40];
```

```
    float salario;
```

```
} Func;
```

```
Func chefe;
```

```
std::cout << "Insira a Idade do chefe: \n";
```

```
std::cin >> chefe.idade;
```

```
std::cout << "Insira o Nome do chefe: \n";
```

```
std::cin >> chefe.nome;
```

```
std::cout << "Insira o Salario do chefe: \n";
```

```
std::cin >> chefe.salario;
```

Estruturas Aninhadas

Estruturas Aninhadas

```
typedef struct
{
    int dia;
    char mes[10];
    int ano
} Data;
```

```
typedef struct
{
    int pecas;
    float preco;
    Data diaVenda; //variavel do tipo struct Data
} Venda;
```

```
int main(){
    Venda A= {20, 115.0, {8, "Dezembro", 2020}};

    std::cout << "Pecas: \n" << A.pecas;
    std::cout << "Preco: \n" << A.preco;
    std::cout << "Data: " << A.diaVenda.dia << " de " <<
    A.diaVenda.mes << " de " << A.diaVenda.ano << "\n";

    system("pause");
    return 0;
}
```


Cuidado!

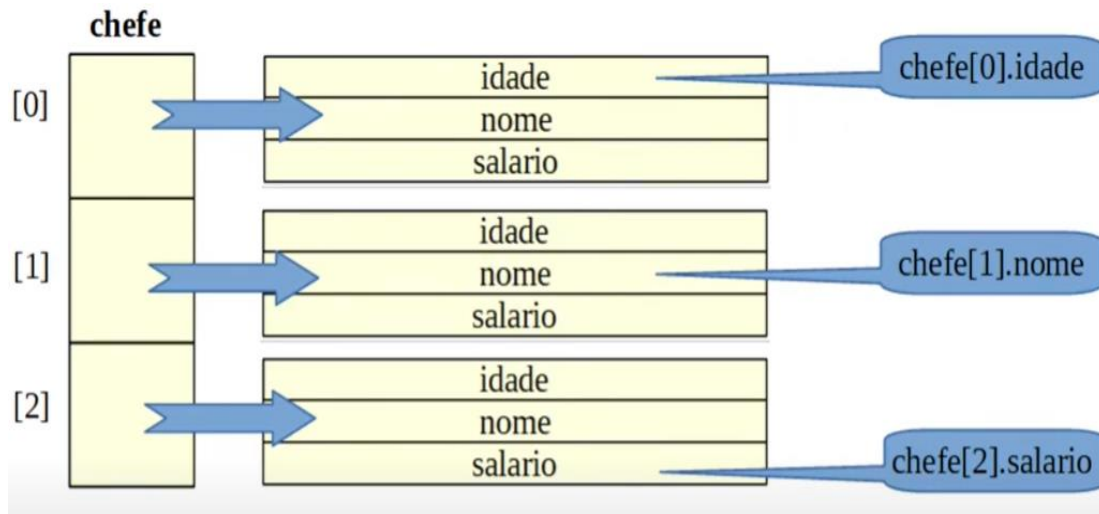
Uma estrutura não pode ter como item uma instância de si mesma, exceto que este seja **um ponteiro (*)**.

```
struct Aluno
{
    int matricula;
    char nome[40];
    int idade;
    struct Aluno aluno1; //ERRO
    struct Aluno *ptr_aluno; //CORRETO
};
```

Vetor de Estruturas

Estruturas Aninhadas

```
typedef struct  
{  
    int idade;  
    char nome[40];  
    float salario;  
} Func;  
  
Func chefe[3];
```



Exercícios

Crie um programa que leia e apresente os dados de 3 funcionários. Crie uma struct que contenha o nome, idade, salário e a data de nascimento. O que se sabe é que a data de nascimento é do tipo Data, ou seja, uma outra struct que contem os seguintes membros: dia e ano, ambos do tipo inteiro; e mês do tipo string. Crie um vetor de struct para registrar os dados dos 3 funcionários.

Defina uma struct para estruturar dados de alunos de uma escola. Dentro dessa struct, crie uma variavel para armazenar o nome do aluno, e outras para armazenar as notas de matemática, física e a média dessas duas notas. Após definir a struct, crie três variáveis do tipo struct que você criou. Preencha os nomes e as notas dos alunos, calculando automaticamente a média deles.

Crie um programa que leia a venda de 10 produtos (use um vetor). De cada produto queremos saber o nome, preço, quantidade de um produto vendido, e mostre também o valor total das vendas. Use structs para estruturar os dados.



C . E . S . A . R

Pessoas impulsionando inovação.
Inovação impulsionando negócios.

